Project Overview:

This executive summary gives a detailed analysis of the scottyab/rootbeer GitHub repository, which is an Android security library designed to detect rooted devices. The objective of this analysis was to examine and track development activity contributor behavior, and file modification patterns using repository mining techniques. By analyzing commit history and source-file changes over time, this report emphasizes key insights relevant to project maintenance, ownership, and potential risks.

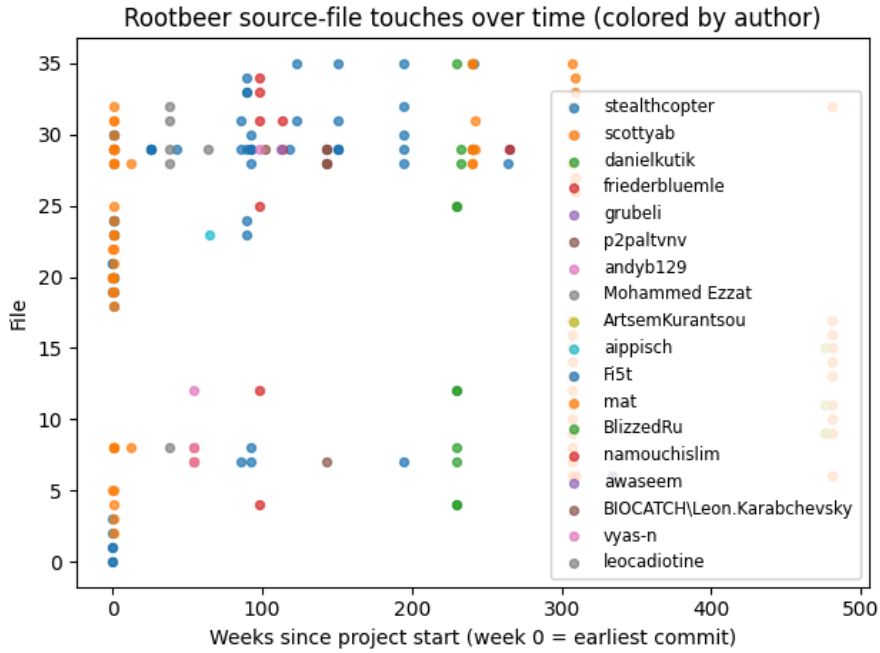
Development Activity and Contributors:

The repository has been maintained over a long time, about 480 weeks from the earliest commit to recent development activity. Analysis of commit data shows that scottyab is the main and most consistent contributor. Illustrating strong ownership and long-term commitment to the project.

The data also shows several secondary contributors appear off and on in the project timeline. They touch a limited number of files over short time spans, which suggests short-term involvement. Possibly doing bug fixes, adding features, or experimental changes rather than doing maintenance.

File Volatility and Ownership Patterns:

Source-file analysis showed 36 unique source files written mainly in Java, Kotlin and C/C++. Some files show a significantly higher modification frequency than others represented by thicker clusters. Especially RootBeer.Java, which stands out as the most frequently modified file, showing it has a crucial role in the project's functionality and updates.
Visualization:



Risk and Recommendations:

From a project management point of view, having a single primary maintainer represents both a strength and a potential risk. While this idea supports consistency and design coherence, it can also create a single point of failure if the main contributor becomes unavailable. So it is recommended that highly active files should get extra documentation and testing support to counter maintenance risk.

Tools and Methods Used:

This analysis was conducted using Git and GitHub repository mining techniques. Key tools and commands included:

- git clone to obtain the repository history
- Branch management using git checkout -b
- GitHub's REST API to retrieve commit, author, and file-level metadata
- Python scripts to automate data collection and filtering
- Matplotlib to visualize file modification activity over time